Proposal: XStore

eXtreme performance in distributed kv-Store for time-series

Lan Nguyen

Data-Intensive Distributed Systems Laboratory
Illinois Institute of Technology

October 18, 2021



ILLINOIS TECH | College of Computing

1. Introduction
2. Existing Solutions
3. Research Problems
4. XStore

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore

Time series is ...

- Sequence of data points

Time series is ...

- Sequence of data points
- Index in time order

Time series is ...

- Sequence of data points
- Index in time order
- Successive measurements over a time interval

Time series is ...

- Sequence of data points
- Index in time order
- Successive measurements over a time interval
- Derive from a single source

Time series use cases

- IoT: Moisture, heart rate, etc.

Time series use cases

- IoT: Moisture, heart rate, etc.
- DevOps: Temperature, users' events, etc.

Time series use cases

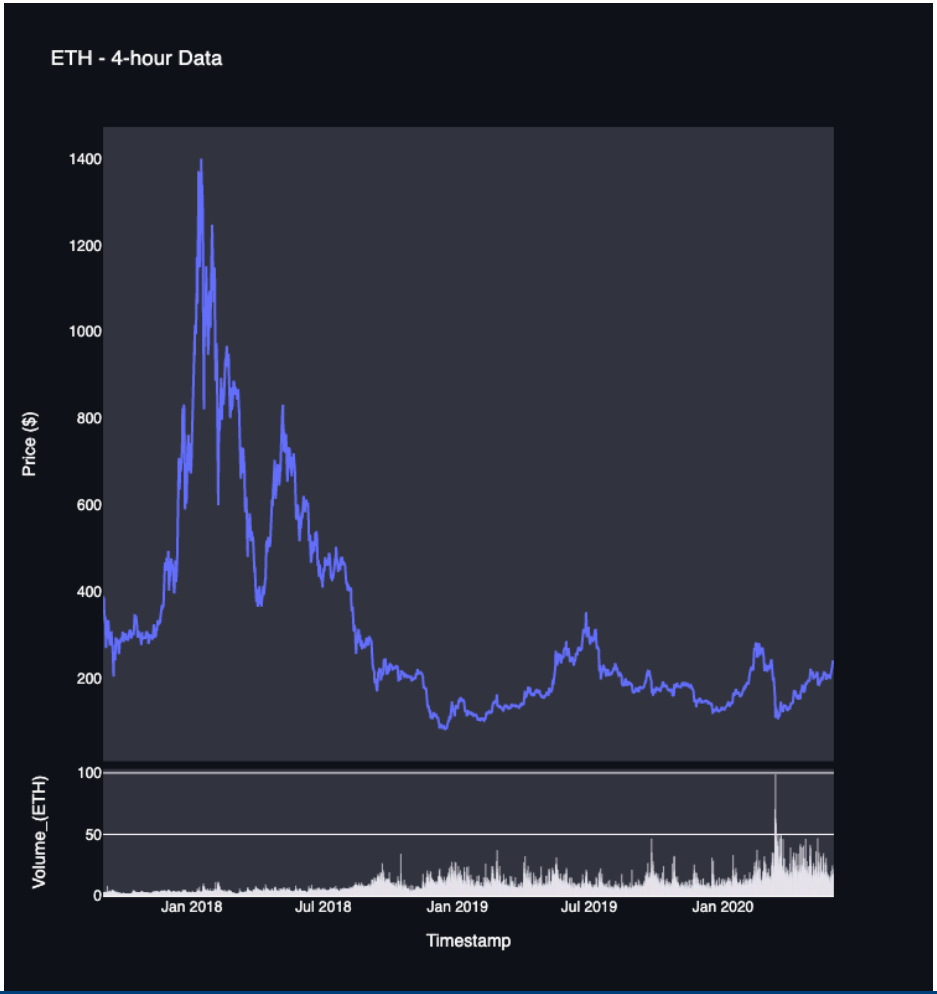
- IoT: Moisture, heart rate, etc.
- DevOps: Temperature, users' events, etc.
- Finance: Backtesting, fraud-detection, etc.

Time series use cases

- IoT: Moisture, heart rate, etc.
- DevOps: Temperature, users' events, etc.
- Finance: Backtesting, fraud-detection, etc.
- Science researches: Nuclear simulations, quantum computing environment etc.

Ethereum spot price: Table & plot

	Timestamp	Open	High	Low	Close	Volume_(ETH)	Volume_(USDT)	Weighted_Price
86745593	1590987594	239.9000	239.9100	239.9000	239.9100	23.3262	5,595.9875	239.9014
86745594	1590987595	239.9200	239.9400	239.9200	239.9400	6.1557	1,476.8986	239.9234
86745595	1590987596	NaN	NaN	NaN	NaN	NaN	NaN	NaN
86745596	1590987597	239.9400	239.9400	239.9400	239.9400	0.0460	11.0276	239.9400
86745597	1590987598	239.9300	239.9300	239.9100	239.9100	1.1750	281.9059	239.9117



1 Introduction

2 Existing Solutions
- DB model engines ranking
- InfluxDB
- MongoDB

3 Research Problems

4 XStore

1 Introduction

2 Existing Solutions
- DB model engines ranking
- InfluxDB
- MongoDB

3 Research Problems

4 XStore

Time series model

Rank Oct 2021	Rank Sep 2021	Rank Oct 2020	DBMS	Database Model	Score Oct 2021	Score Sep 2021	Score Oct 2020
1.	1.	1.	InfluxDB	Time Series, Multi-model	28.52	-0.98	+4.37
2.	2.	2.	Kdb+	Time Series, Multi-model	8.00	-0.13	+0.34
3.	3.	3.	Prometheus	Time Series	6.64	+0.21	+1.31
4.	4.	4.	Graphite	Time Series	5.50	+0.41	+1.14
5.	5.	↑ 6.	TimescaleDB	Time Series, Multi-model	3.96	+0.24	+1.05

Multi-model

Rank Oct 2021	Rank Sep 2021	Rank Oct 2020	DBMS	Database Model	Score Oct 2021	Score Sep 2021	Score Oct 2020
1.	1.	1.	MongoDB	Document, Multi-model	493.55	-2.95	+45.53
2.	2.	2.	Redis	Key-value, Multi-model	171.35	-0.59	+18.07
3.	3.	3.	Teradata	Relational, Multi-model	69.83	+0.15	-5.96
4.	4.	4.	InfluxDB	Time Series, Multi-model	28.52	-0.98	+4.37
5.	5.	5.	Informix	Relational, Multi-model	23.12	+0.17	+0.41

1 Introduction

2 Existing Solutions
- DB model engines ranking
- InfluxDB
- MongoDB

3 Research Problems

4 XStore

InfluxDB

Designed for large and high write loads:

- IoT & Sensor monitoring

InfluxDB

Designed for large and high write loads:

- IoT & Sensor monitoring
- DevOps monitoring

InfluxDB

Designed for large and high write loads:

- IoT & Sensor monitoring
- DevOps monitoring
- Real-time analytics

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()
  - Incur additional disk footprint to maintain data resiliency

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()
  - Incur additional disk footprint to maintain data resiliency
- Cache
  Storing current data points in-memory for further processing

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()
  - Incur additional disk footprint to maintain data resiliency
- Cache
  Storing current data points in-memory for further processing
  - Inefficient use of memory due to storing uncompressed data

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()
  - Incur additional disk footprint to maintain data resiliency
- Cache
  Storing current data points in-memory for further processing
  - Inefficient use of memory due to storing uncompressed data
  - Incur additional computational cost due to data organization by key

InfluxDB engine components

- Write Ahead Log (WAL)
  Ensuring data is durable in case of unexpected failure
  - Computationally expensive due to fsync()
  - Incur additional disk footprint to maintain data resiliency
- Cache
  Storing current data points in-memory for further processing
  - Inefficient use of memory due to storing uncompressed data
  - Incur additional computational cost due to data organization by key
  - Cache miss

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding
- Time Series Index (TSI) Ensuring queries remain fast as data cardinality grows

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding
- Time Series Index (TSI) Ensuring queries remain fast as data cardinality grows
  - Inefficient use of memory due to storing indexes

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding
- Time Series Index (TSI) Ensuring queries remain fast as data cardinality grows
  - Inefficient use of memory due to storing indexes
  - Memory constrained

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding
- Time Series Index (TSI) Ensuring queries remain fast as data cardinality grows
  - Inefficient use of memory due to storing indexes
  - Memory constrained
  - Higher cardinality leads to slower boot time

InfluxDB engine components (Cont.)

- Time-Structured Merge Tree (TSM) Group field values by series key then orders field by time
  - Incur additional computational cost due to encoding
  - Nanoseconds deteriorate performance due to encoding
- Time Series Index (TSI) Ensuring queries remain fast as data cardinality grows
  - Inefficient use of memory due to storing indexes
  - Memory constrained
  - Higher cardinality leads to slower boot time
  - Hashing incurs additional computational costs

1 Introduction

2 Existing Solutions
- DB model engines ranking
- InfluxDB
- MongoDB

3 Research Problems

4 XStore

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- In-Memory Storage Engine Maintain data only in memory

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- In-Memory Storage Engine Maintain data only in memory
  - Non-persistent

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- In-Memory Storage Engine Maintain data only in memory
  - Non-persistent
  - Memory constrained

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- In-Memory Storage Engine Maintain data only in memory
  - Non-persistent
  - Memory constrained
  - Unscalable, especially with large amount of data

MongoDB

Storage engines:

- WiredTiger Storage Engine Group field values by series key then orders field by time
  - Requires snapshot to maintain data resiliency
  - Compression incurs additional computational costs
  - Querying relies on index. Impractical for fine-grained time series data
    1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- In-Memory Storage Engine Maintain data only in memory
  - Non-persistent
  - Memory constrained
  - Unscalable, especially with large amount of data
  - Hashing incurs additional computational costs

1 Introduction

2 Existing Solutions

3 Research Problems
- Existing solutions' problems
- Motivation

4 XStore

1 Introduction

2 Existing Solutions

3 Research Problems
- Existing solutions' problems
- Motivation

4 XStore

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs
- Querying relies on index. Impractical for fine-grained time series data
  1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs
- Querying relies on index. Impractical for fine-grained time series data
  1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- Non-persistent

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs
- Querying relies on index. Impractical for fine-grained time series data
  1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- Non-persistent
- Memory constrained

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs
- Querying relies on index. Impractical for fine-grained time series data
  1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- Non-persistent
- Memory constrained
- Unscalable, especially with large amount of data

Existing solutions' problems

- Incur additional disk footprint to maintain data resiliency
- Requires snapshot to maintain data resiliency
- Compression incurs additional computational costs
- Querying relies on index. Impractical for fine-grained time series data
  1 year data = 31,556,952,000,000,000 nanoseconds or 31 quadrillion indexes
- Non-persistent
- Memory constrained
- Unscalable, especially with large amount of data
- Hashing incurs additional computational costs

1 Introduction

2 Existing Solutions

3 Research Problems
- Existing solutions' problems
- Motivation

4 XStore

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours
- 1 hour = 60 minutes

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours
- 1 hour = 60 minutes
- 1 minute = 60 seconds

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours
- 1 hour = 60 minutes
- 1 minute = 60 seconds
- 1 second = 1000 milliseconds

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours
- 1 hour = 60 minutes
- 1 minute = 60 seconds
- 1 second = 1000 milliseconds
- 1 millisecond = 1000 microseconds

Motivation #1

Time series data contains timestamp field, which is a fixed structure such:
Order of magnitude of 1 year of time:

- 1 year = 12 months
- 1 month $\leq$ 31 days
- 1 day $\leq$ 24 hours
- 1 hour = 60 minutes
- 1 minute = 60 seconds
- 1 second = 1000 milliseconds
- 1 millisecond = 1000 microseconds
- 1 microsecond = 1000 nanoseconds

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp
- Simple yet efficient search/retrieve/delete routines

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp
- Simple yet efficient search/retrieve/delete routines
- Extremely low memory footprint

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp
- Simple yet efficient search/retrieve/delete routines
- Extremely low memory footprint
- Eliminate hashing using time structure exploitation

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp
- Simple yet efficient search/retrieve/delete routines
- Extremely low memory footprint
- Eliminate hashing using time structure exploitation
- Scalable regardless of granularity of time series data

Motivation #2

An idea to exploit the fixed structure of time index to:

- Achieve superior performance, $O(1)$ in search/get/delete given a timestamp
- Simple yet efficient search/retrieve/delete routines
- Extremely low memory footprint
- Eliminate hashing using time structure exploitation
- Scalable regardless of granularity of time series data
- Fault-tolerant

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore
- Indexing fixed structure of time
- Storing data example
- High-level thoughts for hash map
- Supported operations
- Offset algorithm

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore

Indexing fixed structure of time

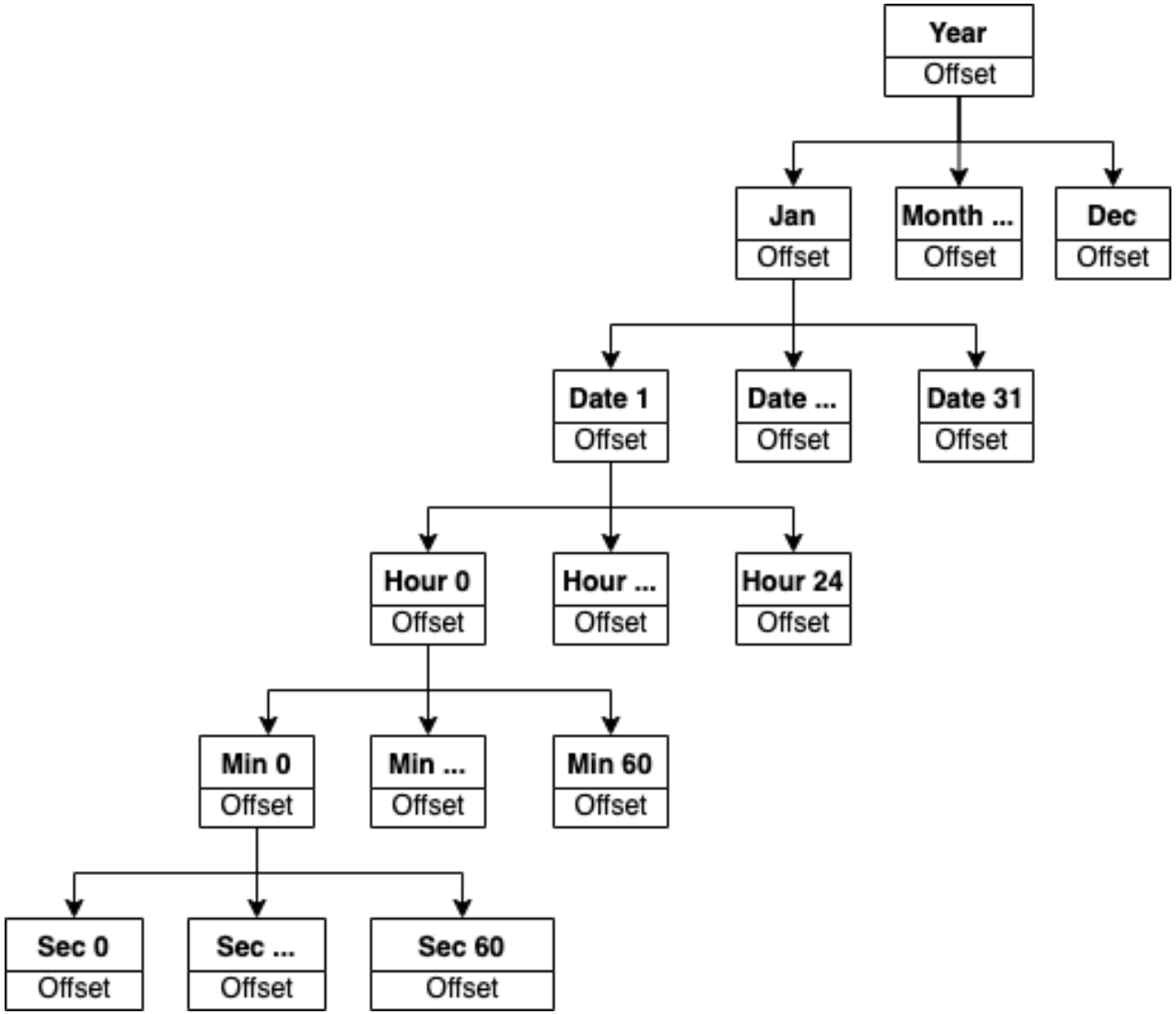
Storing data example

High-level thoughts for hash map

Supported operations

Offset algorithm

Hash map design: Fixed structure of time



1 Introduction

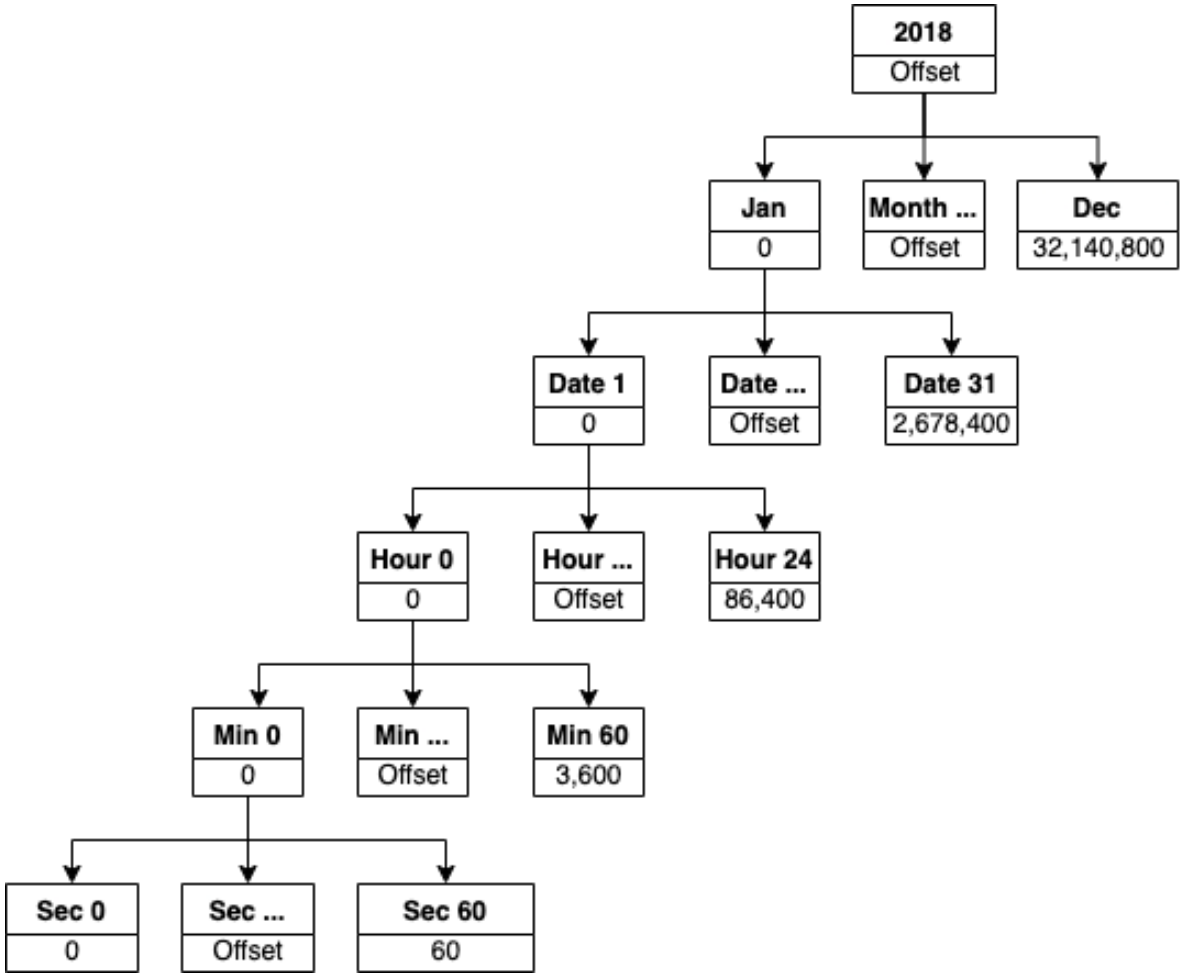
2 Existing Solutions

3 Research Problems

4 XStore
- Indexing fixed structure of time
- Storing data example
- High-level thoughts for hash map
- Supported operations
- Offset algorithm

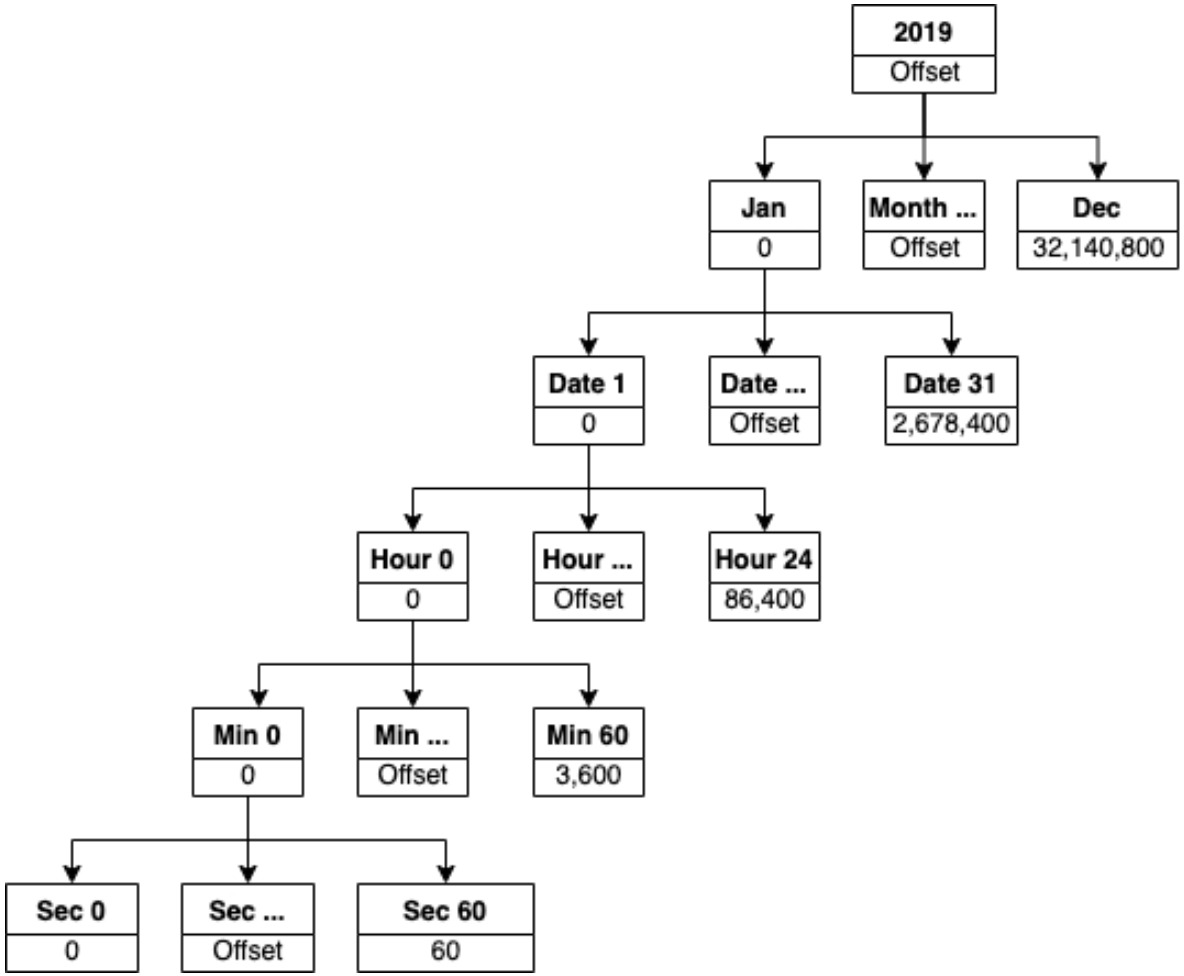
Storing data example

Ingesting 2 years of Ethereum data from 01-01-2018 to 12-31-2019



Storing data example

Each year of data is serialized to binary file format



Storing data example

Offset	Timestamp	Price
0	01/01/2018 00:00:00	3405.50
1	01/01/2018 00:00:01	3405.50
...	...	...
32,140,799	12/31/2018 23:59:58	3405.50
32,140,800	12/31/2018 23:59:59	3405.50

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore
- Indexing fixed structure of time
- Storing data example
- High-level thoughts for hash map
- Supported operations
- Offset algorithm

High-level thoughts for hash map:

- Hash map is always in-memory at initialization

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13
- Each year is stored as binary file format

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13
- Each year is stored as binary file format
- Items in hash map contains offset value for $O(1)$ insert/lookup/delete

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13
- Each year is stored as binary file format
- Items in hash map contains offset value for $O(1)$ insert/lookup/delete
- Insert/lookup/delete operations rely on simple arithmetic calculations to obtain offset

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13
- Each year is stored as binary file format
- Items in hash map contains offset value for $O(1)$ insert/lookup/delete
- Insert/lookup/delete operations rely on simple arithmetic calculations to obtain offset
- Due to small in size, flush hash map to disk frequently

High-level thoughts for hash map:

- Hash map is always in-memory at initialization
- No collision due to the nature of time
  i.e., There can't be Month 13
- Each year is stored as binary file format
- Items in hash map contains offset value for $O(1)$ insert/lookup/delete
- Insert/lookup/delete operations rely on simple arithmetic calculations to obtain offset
- Due to small in size, flush hash map to disk frequently
- Note: Hash map is optional

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore
- Indexing fixed structure of time
- Storing data example
- High-level thoughts for hash map
- Supported operations
- Offset algorithm

Supported operations:

- Insert

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Calculate using the rule of time structure:
     01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
     01/02/2018 00:00:02 yields offset = 86,401 in file 2018.XSTORE

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Calculate using the rule of time structure:
     01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
     01/02/2018 00:00:02 yields offset = 86,401 in file 2018.XSTORE
- Delete

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Calculate using the rule of time structure:
     01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
     01/02/2018 00:00:02 yields offset = 86,401 in file 2018.XSTORE
- Delete
  1. Check hash map if year.XSTORE exists. If yes, proceed

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Calculate using the rule of time structure:
     01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
     01/02/2018 00:00:02 yields offset = 86,401 in file 2018.XSTORE
- Delete
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Nullify data at specific time location in binary file format i.e., 01/01/2018 00:00:01 contains NULL

Supported operations:

- Insert
  1. Check hash map if year.XSTORE exists. If not, proceed
  2. Write data into specific time location to binary file format i.e., 01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
  3. Upon written successful, update hash map
- Get
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Calculate using the rule of time structure:
     01/01/2018 00:00:01 yields offset = 1 in file 2018.XSTORE
     01/02/2018 00:00:02 yields offset = 86,401 in file 2018.XSTORE
- Delete
  1. Check hash map if year.XSTORE exists. If yes, proceed
  2. Nullify data at specific time location in binary file format i.e., 01/01/2018 00:00:01 contains NULL
  3. Upon written successful, update hash map

1 Introduction

2 Existing Solutions

3 Research Problems

4 XStore
- Indexing fixed structure of time
- Storing data example
- High-level thoughts for hash map
- Supported operations
- Offset algorithm

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00
3. Assuming target is 01/02/2018 00:00:02:
   Compute difference = targetDT - baseDT
   Returns: 01/02/2018 00:00:02 - 01/01/2018 00:00:00
   = 86,402 seconds

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00
3. Assuming target is 01/02/2018 00:00:02:
   Compute difference = targetDT - baseDT
   Returns: 01/02/2018 00:00:02 - 01/01/2018 00:00:00
   = 86,402 seconds
4. Offset = 86,402 + sizeOfValue | Explanation:

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00
3. Assuming target is 01/02/2018 00:00:02:
   Compute difference = targetDT - baseDT
   Returns: 01/02/2018 00:00:02 - 01/01/2018 00:00:00
   = 86,402 seconds
4. Offset = 86,402 + sizeOfValue | Explanation:
   - Offset is in seconds due to granularity = 1 second

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00
3. Assuming target is 01/02/2018 00:00:02:
   Compute difference = targetDT - baseDT
   Returns: 01/02/2018 00:00:02 - 01/01/2018 00:00:00
   = 86,402 seconds
4. Offset = 86,402 + sizeOfValue | Explanation:
   - Offset is in seconds due to granularity = 1 second
   - Granularity is defined at initialization

Offset Algorithm

1. Get yearValue from file name
   yearVal = 2018 when 2018.XSTORE
2. Assumption:
   - monthVal, dayVal, hourVal, minVal, secVal starts at 0
   - We have: baseDT = 01/01/2018 00:00:00
3. Assuming target is 01/02/2018 00:00:02:
   Compute difference = targetDT - baseDT
   Returns: 01/02/2018 00:00:02 - 01/01/2018 00:00:00
   = 86,402 seconds
4. Offset = 86,402 + sizeOfValue | Explanation:
   - Offset is in seconds due to granularity = 1 second
   - Granularity is defined at initialization
   - sizeOfValue = Size of value placeholders
     i.e., Open | High | Low | Close

Thank you for your time and patience!